Claude Code vs GitHub Copilot Chat: A Detailed Comparison

A side-by-side guide to help you learn both tools without getting confused. Each section maps the equivalent concept across both platforms.

Quick Reference Table

Purpose	Claude Code	GitHub Copilot
Config folder	`.claude/`	`.github/`
Project context file	`CLAUDE.md`	`copilot-instructions.md`
Specialized capabilities	Skills (`SKILL.md`)	Skills (`SKILL.md`)
Reusable prompts	Slash commands	Prompt files (`.prompt.md`)
Specialized AI personas	Subagents	Custom agents (`.agent.md`)
Always-on cross-agent rules	`CLAUDE.md` (covers this)	`AGENTS.md`
File-scoped rules	Subfolder `CLAUDE.md` files	`.instructions.md` files
External tool integration	MCP servers	MCP servers

---

1. Config Folder: `.claude/` vs `.github/`

The root directory where each tool looks for all customization files.

Claude Code: `.claude/`

- **Project path:** `your-repo/.claude/`
- **Global path:** `~/.claude/`
- **Contains:** `commands/`, `skills/`, `agents/`, settings
- **Note:** `CLAUDE.md` itself sits at repo root, *not* inside `.claude/`
- **Git:** typically committed to repo for team sharing

```
your-repo/
├── CLAUDE.md
└── .claude/
    ├── commands/
    ├── skills/
    └── agents/
```

GitHub Copilot: `.github/`

- **Project path:** `your-repo/.github/`
- **Global path:** `~/.copilot/`
- **Contains:** `agents/`, `skills/`, `instructions/`, `prompts/`, `chatmodes/`
- **Note:** `copilot-instructions.md` lives *inside* `.github/`
- **Git:** committed to repo; also supports org-level via `.github-private` repo

```
your-repo/
├── AGENTS.md
└── .github/
    ├── copilot-instructions.md
    ├── agents/
    ├── skills/
    ├── instructions/
    └── prompts/
```

Key Differences

- Claude Code keeps its main context file (`CLAUDE.md`) at the repo root as a top-level citizen, while all customization machinery lives inside `.claude/`. Copilot puts *everything* inside `.github/` — including its main instructions file.
- The global personal directory: Claude Code uses `~/.claude/` while Copilot uses `~/.copilot/`.
- `.github/` has been a standard GitHub directory for years (it already houses workflows, issue templates, etc.), so Copilot piggybacks on that existing convention. `.claude/` is a brand-new directory that exists solely for Claude Code.

---

2. Project Context: `CLAUDE.md` vs `copilot-instructions.md`

Both serve the same purpose: telling the AI about your project's architecture, conventions, tech stack, and rules.

Claude Code: `CLAUDE.md`

- **Location:** project root, any subfolder, or `~/.claude/`
- **Loaded:** automatically, every conversation
- **Nesting:** subfolder files append to root context
- **Created via:** `/init` command or manually
- **Scope:** global + project + subfolder

GitHub Copilot: `copilot-instructions.md`

- **Location:** `.github/copilot-instructions.md`
- **Loaded:** automatically, every chat request
- **Nesting:** use `.instructions.md` files for scoped rules
- **Created via:** `/init` or `/create-instructions` or manually
- **Scope:** repo + org level

Key Differences

- `CLAUDE.md` is placed directly at the project root (not inside `.claude/`), and you can also have additional ones in subfolders like `frontend/CLAUDE.md`. `copilot-instructions.md` must live inside `.github/`.
- For file-scoped rules in Copilot, you create separate `.instructions.md` files under `.github/instructions/` that can target specific file patterns (like "apply these rules only to `*.tsx` files"). Claude Code handles this via subfolder `CLAUDE.md` files instead.
- `CLAUDE.md` has a dual role — it handles both project context AND behavioral rules. Copilot separates these into `copilot-instructions.md` (context) and `AGENTS.md` (behavioral rules).

---

3. Skills (`SKILL.md`): Same Name, Same Concept, Open Standard

Both tools use `SKILL.md` files inside a named folder. The format is nearly identical: YAML frontmatter (name, description) + markdown instructions + optional scripts and resources. **Agent Skills is now an open standard** that both tools support.

Claude Code Skills

- **Location:** `.claude/skills/<name>/SKILL.md`
- **Global:** `~/.claude/skills/<name>/SKILL.md`
- **Discovery:** description field auto-matched to task
- **Loading:** descriptions always in context; full body loaded on demand
- **Manual invoke:** `/skill-name` (also works as slash command)
- **Arguments:** `$ARGUMENTS` placeholder
- **Tool restriction:** `allowed-tools` frontmatter field
- **Subagent mode:** `context: fork` runs skill in isolated agent
- **Extra frontmatter:** `agent`, `disable-model-invocation`

Copilot Skills

- **Location:** `.github/skills/<name>/SKILL.md`
- **Global:** `~/.copilot/skills/<name>/SKILL.md`
- **Discovery:** description field auto-matched to task
- **Loading:** 3-level progressive (name → body → resources)
- **Manual invoke:** `/skill-name` in chat
- **Arguments:** not explicitly documented
- **Tool restriction:** handled at the agent level, not skill level
- **Subagent mode:** no — skills run in current chat context
- **Cross-compat:** also reads from `.claude/skills/` and `.agents/skills/`

Key Differences

- Claude Code's implementation is richer. It lets you run a skill inside a forked subagent using `context: fork`, restrict which tools the skill can access via `allowed-tools`, and control whether Claude can auto-invoke it with `disable-model-invocation`.
- Copilot's implementation is leaner — it relies on the same discovery-and-load mechanism but delegates tool restrictions to the agent profile (`.agent.md`) rather than the skill itself.

- Copilot reads skills from *three* possible directory paths (`.github/skills/`, `.claude/skills/`, `.agents/skills/`), making skills truly portable. Claude Code currently only reads from its own `.claude/skills/` path.
- The SKILL.md format itself (YAML frontmatter + markdown body + optional scripts in the same directory) is identical across both tools.

---

4. Reusable Prompts: Slash Commands vs Prompt Files

Saved prompt templates you trigger manually — both use `/slash` invocation.

Claude Code: Slash Commands

- **File format:** `.claude/commands/deploy.md`
- **Global:** `~/.claude/commands/deploy.md`
- **Invoke:** `/deploy` in terminal
- **Arguments:** `$ARGUMENTS`, `$1`, `$2`
- **Scoping:** subdirectory-aware (e.g. `commands/frontend/test.md` vs `commands/backend/test.md`)
- **Discovery:** tab autocomplete in terminal
- **Context refs:** no special syntax — just write instructions in English
- **Mode control:** none — runs in current session
- **Can trigger:** subagents, skills, bash scripts

```
# .claude/commands/deploy.md
Deploy $1 to $2 environment.
Run tests first, then build and push.
```

GitHub Copilot: Prompt Files

- **File format:** `.github/prompts/deploy.prompt.md`
- **Global:** user-level prompts in VS Code settings
- **Invoke:** `/deploy` in chat or `#prompt:deploy`
- **Arguments:** not natively supported — describe in prompt body
- **Scoping:** flat directory, no subdirectory routing
- **Discovery:** slash autocomplete in IDE chat panel
- **Context refs:** `#file:path` and markdown links to pull in files
- **Mode control:** `mode: agent` in frontmatter
- **Can trigger:** agent mode, tool use, file references

```
# .github/prompts/deploy.prompt.md
---
mode: agent
description: Deploy the app
---
Run tests, then build and deploy.
Check #file:deploy.config.ts for settings.
```

Key Differences

- Claude Code commands are plain markdown files where the filename becomes the command — no frontmatter needed. They support positional arguments (`$1`, `$2`) and can be organized into subdirectories that scope by working directory (so `/test` invokes a different command depending on whether you're in `/frontend` or `/backend`).
- Copilot prompt files require YAML frontmatter with at least a `description`, and they have a powerful `#file:path` syntax for pulling specific files into the prompt's context. They can also set `mode: agent` to force agent mode when the prompt runs. However, they don't support positional arguments or subdirectory scoping.
- Claude Code slash commands can orchestrate subagents and invoke skills as part of their instructions. Copilot prompt files work more as self-contained prompt injections — they shape the conversation but don't explicitly chain to other agents (that's what handoffs in `.agent.md` files are for).

---

5. Specialized AI Personas: Subagents vs Custom Agents

Claude Code: Subagents

Forked child processes with isolated context.

- **File:** `.claude/agents/*.md`
- **Invoke:** `/agents` or auto-spawned by Claude
- **Context:** fresh, isolated from parent conversation
- **Parallel:** yes, multiple subagents at once
- **Memory:** persistent memory directory (project, user, or local scope)
- **Built-in types:** Explore (read-only), general-purpose, Bash, custom
- **Model:** inherits from parent session

GitHub Copilot: Custom Agents

Persona profiles that reshape the chat agent.

- **File:** `.github/agents/*.agent.md`
- **Invoke:** dropdown picker or `@agent-name` in chat
- **Context:** shares the current chat context
- **Parallel:** no — you switch between agents
- **Handoffs:** chain agents via handoff buttons (e.g. Plan → Implement)
- **Model:** can specify preferred AI model (Claude, GPT, Gemini)
- **Tool restriction:** per-agent via `tools:` field in frontmatter

Key Differences

- Claude Code subagents are **forked processes** — they spin up a separate Claude instance with its own isolated context, do work in parallel, and return results to the parent. This prevents "context pollution" in your main conversation.

- Copilot custom agents are more like **persona switches** — you select an agent from a dropdown and the chat adopts that agent's personality, tools, and instructions. They don't run in parallel; instead, they support "handoffs" where one agent can pass work to another sequentially.
- Claude Code subagents have built-in types like "Explore" (read-only codebase exploration) and support persistent memory across sessions.
- Copilot agents can specify which AI model to use and restrict which tools are available per agent.

---

6. Always-On Cross-Agent Rules: `CLAUDE.md` vs `AGENTS.md`

Persistent rules the AI always follows regardless of which mode or agent is active.

Claude Code: `CLAUDE.md` (Dual Role)

One file handles both project context + behavioral rules.

- **File:** `CLAUDE.md` at project root
- **Scope:** applies to main agent + all subagents inherit project rules
- **Loaded:** automatically, every single conversation
- **Nesting:** subfolder `CLAUDE.md` appends to root
- **Typical contents:** architecture, conventions, build commands, coding standards, behavioral rules — all in one
- **Separate rules file?** No — everything goes in `CLAUDE.md`
- **Philosophy:** single source of truth, less file sprawl

GitHub Copilot: `AGENTS.md` (Dedicated)

Separate file specifically for agent behavioral rules.

- **File:** `AGENTS.md` at repo root
- **Scope:** applies to all Copilot agents (Chat, Coding Agent, CLI)
- **Loaded:** automatically (requires VS Code setting `chat.useAgentsMdFile: true`)
- **Nesting:** nested `AGENTS.md` files for subdirectories
- **Typical contents:** tools allowed, change policies, test requirements, security rules
- **Separate from context?** Yes — `copilot-instructions.md` handles project context
- **Philosophy:** separation of concerns, dedicated files for each purpose

Key Differences

- Claude Code consolidates: your `CLAUDE.md` is simultaneously your project documentation, your coding standards, and your behavioral guardrails. You might have a section titled "Architecture" followed by "Rules" followed by "Build Commands" — all in one file.
- Copilot separates concerns into distinct files: `copilot-instructions.md` is your project context (what this project is, how it's structured), `AGENTS.md` is your behavioral rules (what the agent should and shouldn't do, how to validate changes), and `.instructions.md` files are your file-scoped coding standards.
- In a Copilot repo you might have three or four files doing what a single `CLAUDE.md` does in Claude Code. The tradeoff: more files to manage, but each with a clearer, narrower responsibility.

---

7. File-Scoped Rules: Subfolder `CLAUDE.md` vs `.instructions.md`

Rules that apply only to specific parts of the codebase, not globally.

Claude Code: Subfolder `CLAUDE.md`

- **Mechanism:** drop a `CLAUDE.md` in any subfolder
- **Example:** `frontend/CLAUDE.md`
- **Behavior:** appends to root context (doesn't replace)
- **Targeting:** directory-based — applies when working in that folder
- **File pattern matching:** not supported — purely directory-scoped
- **Format:** same as root `CLAUDE.md` (plain markdown, no frontmatter)

```
your-repo/
├── CLAUDE.md              # root rules
├── frontend/
│   └── CLAUDE.md          # frontend rules
└── backend/
    └── CLAUDE.md          # backend rules
```

GitHub Copilot: `.instructions.md` Files

- **Mechanism:** separate `.instructions.md` files
- **Example:** `.github/instructions/react.instructions.md`
- **Behavior:** applied only when matching files are in context
- **Targeting:** `applyTo` frontmatter with glob patterns
- **File pattern matching:** `applyTo: "**/*.tsx"`
- **Format:** YAML frontmatter + markdown instructions

```
# .github/instructions/react.instructions.md
---
applyTo: "**/*.tsx"
---
Use functional components only.
Always use React.memo for lists.
```

Key Differences

- Copilot's `.instructions.md` files can target files by **glob pattern** — so you can say "apply these React conventions to all `.tsx` files" or "apply these SQL conventions to all `.sql` files." The instructions activate automatically when the agent is working on matching files, and stay dormant otherwise. This is very granular.
- Claude Code's approach is simpler: you create a `CLAUDE.md` in a subfolder, and its rules apply when Claude is working within that directory. There's no file-extension-based targeting. If you need different rules for `.tsx` vs `.css` files in the same directory, you'd have to put everything in one

`CLAUDE.md` and write it as prose ("when working on TypeScript files, do X; when working on CSS, do Y").

---

8. External Tool Integration: MCP Servers

Both use the Model Context Protocol (MCP) for connecting to external tools.

Claude Code MCP

- **Setup:** `claude mcp add <name> <command>`
- **Config stored in:** `.claude/settings.json`
- **Invoke tools:** `/mcp__server__tool`
- **Scoping:** project-level or global
- **Subagent access:** can pass MCP servers to subagents via `mcpServers` frontmatter
- **Transport:** stdio (local processes)

Copilot MCP

- **Setup:** VS Code settings or `.agent.md` frontmatter
- **Config stored in:** `settings.json` or agent profiles
- **Invoke tools:** auto-discovered in agent chat
- **Scoping:** per-agent (via `tools` list in `.agent.md`)
- **Agent access:** per-agent tool restriction via `tools:` field
- **Transport:** stdio, SSE, and streamable HTTP

Key Differences

- Claude Code manages MCP servers through its CLI (`claude mcp add`) and stores configuration in `.claude/settings.json`. You can pass MCP servers to subagents explicitly.
- Copilot manages MCP through VS Code's settings UI or directly in `.agent.md` files — which means you can create an agent that bundles specific MCP tools as part of its identity (for example, a "database-agent" that comes with the PostgreSQL MCP server pre-attached).
- Copilot supports more transport types including SSE and streamable HTTP, while Claude Code primarily uses stdio for local processes.

Summary: Design Philosophy

Aspect	Claude Code	GitHub Copilot
Interface	CLI-first (terminal)	IDE-first (VS Code, JetBrains, etc.)
File organization	Fewer files, consolidated	More specialized files, separated concerns
Project context	One `CLAUDE.md` does it all	Split across `copilot-instructions.md`, `AGENTS.md`, `.instructions.md`
Agent model	Forked subprocesses (parallel, isolated)	Persona switching (sequential, shared context)

Aspect	Claude Code	GitHub Copilot
Extensibility	Skills + commands + subagents	Skills + prompts + agents + instructions
Convergence point	Agent Skills open standard (`SKILL.md`)	Agent Skills open standard (`SKILL.md`)

Both approaches reach the same destination — a customized AI coding assistant — just with different organizational philosophies. Claude Code consolidates under one umbrella; Copilot spreads into more specialized files. The Agent Skills standard is where they meet in the middle, making skills portable across both tools.